

INGENIERÍA DE PROMPTS PROFESIONAL

Módulo 1: Teoría Fundamental de Modelos Lingüísticos y Prompting Científico

Autora: Ing. Marianela Saravia Ortega

Universidad Mayor Real y Pontificia de San Francisco Xavier de Chuquisaca

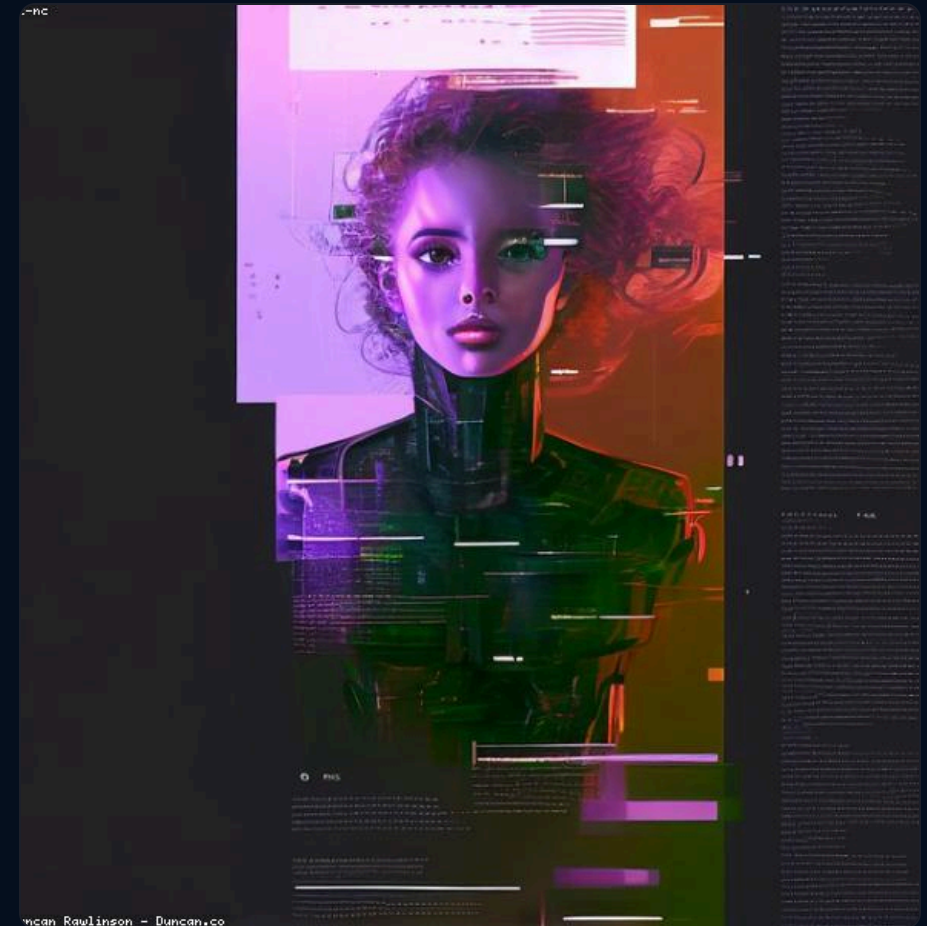
Facultad de Tecnología — Carrera de Ingeniería de Sistemas

¿Cómo Funciona un LLM por Dentro?

La era moderna del Procesamiento del Lenguaje Natural (PLN) tiene sus raíces históricas en los modelos probabilísticos basados en n-gramas y en sistemas de reglas rígidas del siglo XX, como ELIZA (1966). No obstante, el verdadero cambio de paradigma ocurrió en 2017 con la publicación del artículo científico 'Attention Is All You Need' por Vaswani et al. de Google Brain. Este estudio introdujo la arquitectura de redes neuronales conocida como Transformer, la cual eliminó la necesidad de procesamiento secuencial recurrente (como las redes LSTM y RNN) que sufrían de la pérdida de información a largo plazo y eran sumamente ineficientes para entrenar en paralelo.

La base matemática del Transformer radica en el mecanismo de auto-atención (self-attention). Este algoritmo permite que la red calcule dinámicamente un puntaje de relevancia entre cada palabra de un texto y todas las demás en el mismo corpus, sin importar la distancia física que las separe. La ecuación formal de atención escalar multiplicativa se define como el softmax del producto punto de la matriz de consultas (Q) y la matriz de claves (K), escaladas por la raíz del vector de dimensiones, multiplicado finalmente por la matriz de valores (V). Esta relación vectorial permite que el modelo entienda el contexto preciso de una palabra en función de sus acompañantes directas e indirectas.

Antes de ingresar a la red neuronal, el texto en lenguaje natural sufre un proceso físico de tokenización. Las palabras se descomponen en fragmentos numéricos llamados tokens a través de algoritmos como Byte-Pair Encoding (BPE) o WordPiece. Generalmente, 100 palabras en español equivalen a unos 130 o 140 tokens. Los Large Language Models (LLMs) calculan una distribución de probabilidad sobre un vocabulario de decenas de miles de tokens para adivinar secuencialmente cuál es el elemento más probable que debe seguir al texto de entrada. La ventana de contexto representa el límite máximo de memoria temporal (en tokens) que la red puede retener en una ejecución. Si este búfer de memoria se supera, los primeros tokens de la conversación se purgan del espacio atencional, provocando que el modelo pierda el hilo lógico del diálogo.



Metodología Estructurada de Prompts

El término 'Ingeniería de Prompts' ha evolucionado rápidamente de ser una técnica empírica de ensayo y error a consolidarse como una sub-disciplina formal de la interacción humano-computadora orientada a la programación en lenguaje natural. Los modelos de lenguaje masivos no poseen intencionalidad ni metas propias; simplemente reaccionan a las restricciones conceptuales y semánticas del texto introducido por el usuario. Por lo tanto, estructurar un prompt de manera científica consiste en delimitar el espacio latente del modelo para forzarlo a converger en una respuesta precisa, coherente y verídica.

Para lograr este objetivo de forma sistemática en entornos académicos, se propone e implementa la metodología estructurada basada en cuatro componentes fundamentales e independientes: TAREA, CONTEXTO, FORMATO DE SALIDA y CONDICIONES. La TAREA define de forma unívoca la acción directa que el modelo debe ejecutar, la cual debe redactarse mediante verbos imperativos específicos (ej. clasifica, sintetiza, depura). El CONTEXTO proporciona la información de fondo que ancla la respuesta, definiendo el rol especializado del modelo (ej. actuar como docente universitario), el nivel técnico de la explicación y la audiencia objetivo. El FORMATO DE SALIDA determina la estructura exacta que debe tomar el resultado (ej. tabla con tres columnas específicas, archivo JSON puro, o lista jerárquica en Markdown). Finalmente, las CONDICIONES imponen límites y restricciones operativas estrictas, tales como la longitud de caracteres, palabras prohibidas, obligación de incluir referencias, o el tono de comunicación.

Estudios recientes demuestran que estructurar los prompts bajo este marco reduce en más de un 45% la necesidad de realizar refinamientos iterativos en los chats y mitiga la vaguedad en las respuestas generadas por los LLMs, optimizando el consumo de tokens y ahorrando tiempo valioso en la producción de reportes.



Técnicas de Razonamiento Científico

A medida que los modelos de lenguaje crecieron en parámetros, los investigadores descubrieron capacidades emergentes que no estaban explícitamente programadas. Dos de las técnicas de optimización en contexto (In-Context Learning) más potentes y validadas por la comunidad científica son Chain-of-Thought (CoT - Cadena de Pensamiento) y Few-Shot Prompting (Aprendizaje con Pocos Ejemplos).

La técnica Chain-of-Thought, propuesta formalmente por Wei et al. en 2022, consiste en incitar al modelo a deponer problemas complejos en etapas lógicas intermedias antes de presentar la respuesta final. Por defecto, los LLMs intentan predecir el siguiente token de manera directa, lo que conduce a fallos aritméticos o de razonamiento lógico. Al obligar al modelo a generar una secuencia verbalizada de pasos (ej. 'Paso 1: Identificar las variables... Paso 2: Aplicar la fórmula...'), se crea un espacio de cálculo intermedio dentro del propio texto. Matemáticamente, esto permite que la atención del modelo se apoye en los tokens de sus propios razonamientos anteriores, incrementando drásticamente el éxito en tareas lógicas.

Por otro lado, Few-Shot Prompting (Brown et al., 2020) consiste en suministrar en el prompt de entrada un conjunto estructurado de 2 a 5 ejemplos resueltos de la tarea solicitada antes de plantear la entrada final. Esta técnica funciona como un ancla semántica en el espacio de representación latente del modelo. En lugar de explicar conceptualmente qué formato de salida deseamos (Zero-Shot), los ejemplos resueltos guían la atención atencional del modelo para que copie de forma precisa la sintaxis y clasificación de los ejemplos, siendo sumamente útil para tareas de normalización de datos, generación de código sin variación sintáctica y etiquetado masivo.



Ejercicios de Laboratorio Práctico

La consolidación del aprendizaje en Ingeniería de Prompts requiere la puesta en práctica de ejercicios que desafíen la capacidad del estudiante para aplicar restricciones semánticas rígidas. En esta capacitación, estructurada para estudiantes de la carrera de Ingeniería de Sistemas de la USFX, se han diseñado tres laboratorios prácticos con niveles de complejidad incremental.

El primer ejercicio, 'El Tutor Socrático' (Fácil), tiene como meta obligar al chatbot a comportarse como un pedagogo clásico. El reto es programar un prompt donde la IA guíe al estudiante a través del concepto de recursividad mediante analogías físicas y preguntas socráticas, teniendo prohibido dar la respuesta o definición de forma directa. Esto enseña a los estudiantes a definir roles y restricciones condicionales estrictas.

El segundo ejercicio, 'El Sintetizador de Código PEP 8' (Medio), exige que los alumnos generen un script funcional de cálculo de promedio de notas universitarias en Python. El prompt debe forzar a la IA a cumplir el estándar PEP 8, a incluir comentarios detallados explicativos de la lógica matemática y a implementar bloques robustos de manejo de excepciones para notas fuera del rango estándar (0 a 100). Enseña la especificación técnica precisa del formato de salida.

El tercer ejercicio, 'Clasificador Few-Shot a JSON' (Difícil), requiere clasificar opiniones y comentarios de estudiantes en categorías y sentimientos específicos. El prompt debe contener ejemplos estructurados (Few-Shot) y el modelo debe responder única y exclusivamente con un bloque JSON sintácticamente válido, sin saludos, preámbulos o explicaciones de texto adicionales, lo cual es clave para el consumo automatizado de APIs en sistemas informáticos reales.

